동적 라우팅, params, searchParams 샘플 가이드

- 날짜: 2026-10-30
- 대상: Next.js 16.4.3, `lgcns-fe-next`
- 설정: `next.config.ts`의 `cacheComponents: true`, `partialPrefetching: true`
- 블로그와 상품의 `params` 읽기는 현재 소스다. 둘 다 페이지는 프로미스를 `<Suspense>` 안으로 넘기고, 자식이 `await`한다. `searchParams` 샘플은 문서용이다. 이 저장소 페이지는 `searchParams`를 아직 받지 않는다.

URL은 경로와 쿼리로 나뉜다.

```
https://호스트/blog/name?q=cat&page=2
                 └경로┘ └────쿼리────┘
                 params      searchParams
```

`params`는 폴더 이름의 대괄호가 잡아 내는 경로 조각이다. `searchParams`는 `?` 뒤의 쿼리다. Next.js 15부터 둘 다 페이지 props에서 프로미스다. Next.js 14까지는 동기 객체였다. 값을 쓰려면 `await`하거나, 클라이언트 컴포넌트에서는 React `use`로 푼다.

1. 경로 샘플. params

동적 세그먼트 문서는 `app` 폴더 이름이 URL과 어떻게 대응하는지 적는다.

폴더	이 저장소	URL 예	await 후 값
`[blogId]`	`app/(main)/blog/[blogId]/page.tsx`	`/blog/name`	`{ blogId: "name" }`
`[category]/[item]`	없음	`/shop/1/2`	`{ category: "1", item: "2" }`
`[...productCategoryName]`	`app/(main)/product/[...productCategoryName]/page.tsx`	`/product/laptop/gram`	`{ productCategoryName: ["laptop", "gram"] }`
`[[...slug]]`	없음	`/product`	파라미터 없으면 `undefined`

`(main)`은 라우트 그룹이라 URL에 없다. `/blog/a/b`는 `[blogId]` 한 칸에 맞지 않는다. `[...productCategoryName]`은 `/product`만 있는 주소에 맞지 않는다. 그 주소까지 열려면 폴더가 `[[...productCategoryName]]`이어야 하고, 그때 값은 `string[] | undefined`다.

1.1 한 칸. 블로그

현재 `page.tsx`는 프로미스를 그대로 넘긴다. `component/templates/BlogContent.tsx`가 푼다.

```
export default function BlogPage({
  params,
}: {
  params: Promise<{ blogId: string }>
}) {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <BlogContent params={params} />
    </Suspense>
  )
}
```

```
export default async function BlogContent({
  params,
}: {
  params: Promise<{ blogId: string }>
}) {
  const { blogId } = await params
  return (
    <div>
      <h1>Blog Page</h1>
      <p>{blogId}</p>
    </div>
  )
}
```

`/blog/name`의 문단은 `name`이다. `blogId` 타입은 `string`이다.

1.2 catch-all. 상품

`app/(main)/product/[...productCategoryName]/page.tsx`는 동기 함수다. 배열은 `component/templates/ProductContent.tsx`가 푼다.

```
export default function ProductPage({
  params,
}: {
  params: Promise<{ productCategoryName: string[] }>
}) {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <ProductContent params={params} />
    </Suspense>
  )
}
```

```
export default async function ProductContent({
  params,
}: {
  params: Promise<{ productCategoryName: string[] }>
}) {
  const { productCategoryName } = await params
  console.log(productCategoryName.join("/"))
  return (
    <div>
      <CustomTitle TitleType="h1" className="text-2xl font-bold">
        {productCategoryName.join("/")}
      </CustomTitle>
      <p>{productCategoryName}</p>
    </div>
  )
}
```

URL	배열	join("/")
/product/laptop	["laptop"]	laptop
/product/laptop/gram	["laptop", "gram"]	laptop/gram

`console.log`는 서버 컴포넌트가 렌더될 때 서버 터미널에 찍힌다. `await`는 `<Suspense>` 안이다. 페이지 함수는 `Loading...`을 먼저 반환하고, 제목은 배열이 정해진 뒤 이어 붙는다.

2. 쿼리 샘플. searchParams

페이지 문서의 표다. 값은 항상 문자열이다. 숫자처럼 보여도 `"2"`다. 같은 키가 반복되면 배열이다. 없는 키는 `undefined`다. 이 객체는 `URLSearchParams` 인스턴스가 아니다. `.get()`은 없다.

URL	await searchParams
/blog/name?q=cat	{ q: "cat" }
/blog/name?q=cat&page=2	{ q: "cat", page: "2" }
/blog/name?tag=a&tag=b	{ tag: ["a", "b"] }
/blog/name	{}

공식 필터 예는 기본값을 구조 분해에서 준다.

```
const { page = "1", sort = "asc", query = "" } = await searchParams
```

`/blog/name`처럼 쿼리가 없으면 `page`는 `"1"`, `sort`는 `"asc"`, `query`는 `""`다. `/blog/name?page=2`이면 `page`만 `"2"`로 바뀐다.

이 저장소에는 그 코드를 받은 페이지가 없다. `cacheComponents`가 켜진 이 프로젝트에서 블로그에 붙인다면, `params`와 같이 경계 안에서 풀어야 한다. 아래는 문서용 샘플이다.

```
import { Suspense } from "react"

export default function BlogPage({
  params,
  searchParams,
}: {
  params: Promise<{ blogId: string }>
  searchParams: Promise<{
    q?: string
    page?: string
    tag?: string | string[]
  }>
}) {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <BlogContent params={params} searchParams={searchParams} />
    </Suspense>
  )
}

async function BlogContent({
  params,
  searchParams,
}: {
  params: Promise<{ blogId: string }>
  searchParams: Promise<{
    q?: string
    page?: string
    tag?: string | string[]
  }>
}) {
  const { blogId } = await params
  const { q = "", page = "1", tag } = await searchParams
  const tags = tag === undefined ? [] : Array.isArray(tag) ? tag : [tag]
```

```
  return (
    <div>
      <h1>Blog Page</h1>
      <p>{blogId}</p>
      <p>query: {q}</p>
      <p>page: {page}</p>
      <p>tags: {tags.join(",")}</p>
    </div>
  )
}
```

`/blog/name?q=cat&page=2&tag=a&tag=b`이면 화면 값은 `blogId` `name`, query `cat`, page `2`, tags `a,b`다.

클라이언트 컴포넌트 페이지는 `async`가 아니므로 `use`로 푼다. 페이지 문서의 예다.

```
"use client"
import { use } from "react"

export default function Page({
  searchParams,
}: {
  searchParams: Promise<{ q?: string }>
}) {
  const { q = "" } = use(searchParams)
  return <p>{q}</p>
}
```

`useSearchParams()` 훅은 쿼리를 경로와 상관없이 요청 시점에만 알 수 있어서, 클라이언트 컴포넌트에서 쓰면 항상 `<Suspense>`가 필요하다. 이 저장소 네비게이션은 그 훅을 쓰지 않는다.

3. params와 searchParams를 나란히 보면

	params	**searchParams**
위치	경로. `/blog/name`의 `name`	쿼리. `?q=cat`의 `q`
정하는 곳	`app` 폴더의 `[이름]`, `[...이름]`	URL을 여는 쪽. 폴더 이름과 무관
타입	세그먼트마다 `string` 또는 `string[]`	키마다 `string`, 반복 키는 `string[]`, 없으면 `undefined`
이 저장소	블로그·상품 모두 경계 안	아직 사용하지 않음
프로미스	예. `await` 또는 `use`	예. `await` 또는 `use`
Cache Components	경계 밖에서 `await`하면 프리렌더가 막힘	같은 검사. 요청 전에 값을 알 수 없음

둘을 한 페이지에서 같이 쓸 수 있다. `/product/laptop/gram?sort=asc`이면 경로는 배열 `["laptop","gram"]`이고, 쿼리는 `{ sort: "asc" }`다. 상품 페이지는 지금 쿼리를 읽지 않는다.

4. 로딩 시점

브라우저용 `<script src>`는 이 페이지들에 없다. Next가 서버에서 페이지 함수를 실행한다.

1. `cacheComponents`와 `partialPrefetching`이 켜져 있어, 개발 서버는 정적 껍데기를 먼저 만들 수 있는지 본다.
2. `params`와 `searchParams`는 요청 URL이 있어야 한다. 경계 밖에서 `await`하면 그 세그먼트 프리렌더가 막힌다. 메시지는 "Next.js encountered runtime data during prerendering."이다.
3. 블로그는 페이지가 `<Suspense fallback={<p>Loading...</p>}>`를 바로 반환하고, `BlogContent`만 `await params`를 한다.

4. 상품도 같다. 페이지는 `ProductContent`만 감싸고, `await params`는 그 컴포넌트 안에 있다. 쿼리를 나중에 붙일 때도 그 `await`를 같은 경계 안으로 둔다.
5. `export const instant = false`는 두 파일 모두 없다. 그 설정은 블로킹을 허용할 뿐 쿼리를 읽어 주지는 않는다.

5. 공식 문서

- page의 params, searchParams. 로컬 `node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/page.md`
- Dynamic Routes
- Await params inside Suspense
- blocking-prerender-runtime
- useSearchParams
- React use
- MDN URL parameters